BANKITIS ONLINE BANKING SYSTEM

COMPREHENSIVE SOFTWARE ARCHITECTURE, MODULES & UML SPECIFICATION

Academic & Technical Evaluation Master Document • Pure React 19 / Cloudflare / Supabase PostgreSQL

1. Executive System Overview

Bankitis is a high-reliability, enterprise-modeled online banking and peer-to-peer (P2P) payment application. It features real-time recipient name verification, strict daily and per-transaction limits, dedicated 6-digit transaction PIN security with automated lockout, dual-format statement generation (PDF/CSV), personal spending analytics, and live synchronization with cloud PostgreSQL (Supabase).

2. Functional Business Modules (User-Facing Subsystems)

Module Name	Primary Component	Functional Capabilities & Operational Rules
1. Authentication & Onboarding	*AuthTabs.jsx*	Customer sign-in, registration, instant account number generation (ACC1000...), custom UPI handle setup (@bank), and 6-digit PIN initialization.
2. Account & Balance Hub	*BalanceCard.jsx*	Available balance display, eye toggle for masked account numbers, live status tags (Active, Frozen, PIN Locked), and remaining daily limit progress bar.
3. UPI Funds Transfer Engine	*SendMoneyModal.jsx*	Multi-step transfer flow, pre-transfer name resolution (Question 42), bounds check (₹1 - ₹1,00,000), 6-digit PIN validation, and atomic
4. Beneficiary Management	*BeneficiariesModal.jsx*	Add/manage trusted peers without PIN entry, automatic recipient verification before saving, customizable nicknames, and 1-click pay.
5. Customer Directory Search	*DirectoryModal.jsx*	Global registered user search by UPI handle or account number; 1-click selection to immediately prefill the transfer wizard.
6. Transaction Passbook	*TransactionHistory.jsx*	Unlimited passbook with multi-dimensional filtering (Debits vs Credits, Categories, Status, keyword search) and dynamic counterparty resolution.
7. Financial Statement Export	*bankingEngine.js*	Dual statement generators: Formal branded PDF statement with bank header and watermark (jsPDF) + RFC 4180 CSV spreadsheet export (PapaParse).
8. Personal Spending Analytics	*SpendingAnalytics.jsx*	Visual cash flow summary (Money In vs. Money Out) and percentage distribution across categories (Food, Bills, Utilities, Shopping, Entertainment).
9. Notification Center	*Navbar.jsx / App.jsx*	Real-time in-app notification drawer with unread counter for credit receipts, debit alerts, and security login notifications.
10. Admin Executive Console	*AdminClient.jsx*	Dedicated admin portal (/admin) for bank reserves overview, 1-click account freeze/unfreeze, PIN lockout clearance, and manual balance adjustments.
11. Security Audit Trail	*AuditLogView.jsx*	Immutable chronological security log recording logins, transfers, freeze operations, and balance adjustments with reference IDs and timestamps.
12. Cloud Database Connector	*DatabaseSettingsModal.jsx*	In-app connection manager enabling live synchronization with cloud PostgreSQL (Supabase), connection status testing, and RLS management.

3. Core Architecture, Persistence Modules & Technical Stack

A. Core Engine & Technical Modules

Module / Technology	File / Package	Role & Technical Implementation
Core Banking State Engine	*src/lib/bankingEngine.js*	Pure JavaScript deterministic state machine enforcing financial consistency, atomic debit/credit operations, daily limit tracking, and PIN lockout logic.
Supabase Cloud Client	*src/lib/supabaseClient.js*	Initializes @supabase/supabase-js, managing live asynchronous queries, table updates, and connection testing between frontend and cloud database.
PostgreSQL DDL Schema	*supabase_schema.sql*	Complete relational schema defining tables (users, accounts, transactions, beneficiaries, notifications, audit_logs) and Row-Level Security (RLS) policies.
UI Component Framework	*React 19 + React DOM*	Powers reactive single-page components, modals, accessible form states, and instant interactive state updates.
Build & Bundling Engine	*Vite 6*	Provides next-generation build toolchain, asset optimization, tree shaking, and hot module replacement with sub-second reload times.
Styling Design System	*Tailwind CSS v4*	High-performance utility-first CSS framework delivering minimal, responsive, mobile-first fintech interface components.
Vector PDF Generator	*jsPDF & AutoTable*	Enables in-browser vector PDF creation, custom fonts, auto-paginated financial tables, and bank watermarks without backend overhead.
CSV Parser & Serializer	*PapaParse*	RFC 4180 compliant delimited spreadsheet formatting for instant transactional passbook CSV exports.
Iconography Suite	*Lucide React*	Modern, lightweight SVG icons embedded throughout navigation, balance cards, status badges, and action dialogs.

B. Enterprise 3-Tier System Architecture

Architectural Tier	Technology Stack	Responsibilities & Operations
Tier 1: Presentation Layer (Browser Client)	*Cloudflare Pages / React 19 / Tailwind CSS*	Delivered via global edge CDN. Houses the customer dashboard, send money wizard, beneficiary management, admin console (/admin), and responsive mobile views.
Tier 2: Business Logic & Application Layer	*Banking Engine / Controller / Auth*	Enforces strict validation rules: pre-transfer name resolution, amount limits (₹1 - ₹1,00,000), daily ceiling (₹2,00,000), 6-dig lockout timers.
Tier 3: Data & Persistence Layer	*Cloud PostgreSQL (Supabase) + LocalStorage*	Primary: Cloud-hosted PostgreSQL with real-time relational tables (users, accounts, transactions, audit_logs). Fallback: Sandboxed browser localStorage (bankitis_data_v4).

C. Dedicated UML Diagram Source Files (.puml)

Diagram Name	Dedicated Source File	Supported UML Tools
UML Class Diagram	*diagrams/ClassDiagram.puml*	StarUML, Rational Rose, Visual Paradigm, PlantText
UML Activity Diagram	*diagrams/ActivityDiagram.puml*	StarUML, PlantText (Workflow Viva Evaluation)
3-Tier System Architecture	*diagrams/SystemArchitecture.puml*	StarUML Component Diagram, PlantText, Reports
UML Use Case Diagram	*diagrams/UseCaseDiagram.puml*	StarUML, Rational Rose Use-Case View
UML Sequence Diagram	*diagrams/SequenceDiagram.puml*	StarUML, Rational Rose Sequence View
Combined All Diagrams	*diagrams/ALL_UML_DIAGRAMS.puml*	Instant 1-Click Paste for StarUML PlantUML extension

4. UML Class Diagram & Use Case Specifications

A. UML Class Diagram - Domain Entities, Attributes & Operations

Class Entity	Attributes & Data Types	Methods & Operations
User	+ id: String + fullName: String + email: String + phoneNumber: String - passwordHash: String + role: Role (CUSTOMER | ADMIN)	+ register(data: DTO): User + login(credentials): Session + updateProfile(data): Boolean
Account	+ id: String + userId: String [FK] + accountNumber: String + upiId: String - upiPinHash: String + failedPinCount: Integer + pinLockedUntil: DateTime + balance: Decimal + dailyLimit: Decimal + dailySpent: Decimal + status: AccountStatus	+ checkBalance(amount): Boolean + verifyPin(inputPin): Boolean + lockPin(durationMinutes): Void + unlockPin(): Void + updateBalance(delta): Decimal + checkDailyLimit(amount): Boolean
Transaction	+ id: String + referenceNumber: String + senderAccountId: String [FK] + receiverAccountId: String [FK] + amount: Decimal + fee: Decimal + status: TransactionStatus + category: TransactionCategory + senderBalanceAfter: Decimal + receiverBalanceAfter: Decimal + createdAt: DateTime	+ executeAtomicTransfer(): Result + generateReferenceNumber(): String + validateBounds(): Boolean
Beneficiary	+ id: String + ownerId: String [FK] + beneficiaryName: String + beneficiaryUpi: String + accountNumber: String + nickname: String	+ validateRecipient(): Boolean + saveBeneficiary(): Beneficiary + deleteBeneficiary(): Boolean
Notification	+ id: String + userId: String [FK] + title: String + message: String + type: NotificationType + isRead: Boolean + createdAt: DateTime	+ markAsRead(): Void + dispatchAlert(): Void
AuditLog	+ id: String + userId: String [FK] + action: String + details: String / JSON + createdAt: DateTime	+ recordEvent(action, details): Void + queryAuditTrail(filters): List

B. UML Use Case Diagram - Actors & Use Case Interactions

Actor	Associated Use Cases	System Role & Operational Scope
Bank Customer (Actor)	Authenticate / Sign In View Available Balance & Masked Account Send Money via UPI Manage Saved Beneficiaries Search Registered Customer Directory View Transaction Passbook Download Statements (PDF / CSV) View Spending Analytics Receive In-App Notifications	Primary actor who transacts funds, manages peer accounts, inspects personal statements, and configures account settings.
Bank Administrator (Actor)	Authenticate to Executive Portal (/admin) Monitor Total System Liquidity & Volume Freeze / Unfreeze Customer Accounts Clear Security PIN Lockouts Perform Manual Balance Adjustments Inspect Central Transaction Ledger Audit Complete Security Event Log	Administrative actor with elevated privileges to safeguard banking operations, manage fraud risks, and perform audit reviews.

Actor	Associated Use Cases	System Role & Operational Scope
Core Banking Engine (System)	Resolve Recipient Name (<<include>>) Authorize 6-Digit PIN (<<include>>) Check Daily Limits (<<include>>) Execute Atomic Debit/Credit Commit Record Immutable AuditLog Entry	Automated backend system executing atomic ACID guarantees, enforcing transaction bounds, and dispatching transaction receipts.

5. UML Workflow Diagrams & Security Enforcement

A. UML Activity Diagram - Money Transfer Execution Workflow

Workflow Step	System Action & Security Enforcement
Step 1: Recipient Input	Customer enters recipient UPI ID (e.g. recipient@bank) or picks from saved beneficiaries.
Step 2: Recipient Resolution	Asynchronous query verifies account exists and is ACTIVE. Rejects self-transfers immediately.
Step 3: Name Verification	Compliance check (Question 42): Displays verified registered name in green before confirmation.
Step 4: Limits Validation	Verifies: ₹1.00 <= Amount <= ₹1,00,000.00 per transaction; Daily utilized + A
Step 5: Balance Validation	Verifies available balance >= transfer amount. Displays insufficient funds error if unsatisfied.
Step 6: PIN Authentication	Customer enters 6-digit PIN. 3 consecutive failed attempts trigger automated 15-minute security lockout.
Step 7: Atomic Debit & Credit	Deducts transfer amount from sender and credits recipient in a single deterministic atomic commit.
Step 8: Ledger & Audit Commit	Generates TXN-YYYYMMDD-XXXXXX reference number, inserts transaction row, and records AuditLog.
Step 9: Receipts & Alerts	Dispatches in-app credit/debit notifications and displays modal success receipt to the customer.

B. UML Sequence Diagram - Interaction Sequence (P2P Transfer)

Sequence Message Call	Interaction Details & Payload
1. Customer -> Client UI	Inputs recipient UPI handle (recipient@bank).
2. Client UI -> Banking Engine	Calls resolveRecipientUPI(upiId).
3. Banking Engine -> Database	Executes account lookup where upi_id = recipient@bank.
4. Database -> Client UI	Returns { valid: true, name: "Recipient Account" }. Renders green verified badge.
5. Customer -> Client UI	Enters amount (e.g. ₹500), category, note, and 6-digit PIN.
6. Client UI -> Banking Engine	Calls executeMoneyTransfer(recipient, amount, pin, note).
7. Banking Engine -> Security	Verifies PIN hash, checks balance, and validates ₹2,00,000 daily limit.
8. Banking Engine -> Database	Executes atomic balance update, inserts transaction record, and writes AuditLog.
9. Banking Engine -> Client UI	Returns { success: true, referenceNumber, updatedBalance }. Renders success receipt.

C. System Rules & Financial Boundary Enforcements

System Parameter	Enforced Operational Specification
Minimum Transaction Limit	INR 1.00 per transfer (strictly enforced in engine & UI)
Maximum Per-Transaction Limit	INR 1,00,000.00 ceiling per individual transfer
Daily Cumulative Limit	INR 2,00,000.00 per 24-hour cycle with live visual utilization gauge
Transaction Fee	INR 0.00 (Zero Transaction Fee)
PIN Security & Lockout	3 consecutive invalid PIN entries trigger an automated 15-minute temporary lockout
Double-Spending Prevention	Deterministic atomic debit/credit operations with single-commit serialization